

UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS, 2027-I

FACULTAD DE CIENCIAS

PROYECTO:

Chat

PROFESOR:

Canek Pelaez Valdez

AYUDANTE DE TEORÍA:

Cynthia Lizbeth Sánchez Urbano

AYUDANTE DE LABORATORIO:

Miriam Torres Bucio

Introduccion

Se debe crear un chat en línea al que multiples clientes puedan acceder y comunicarse entre ellos, se necesitan crear tanto un servidor como un cliente. El servidor se encargará de recibir las conexiones de los clientes, procesar los mensajes que reciba de ellos y actuar conforme a este, además de enviar respuestas a los mensajes y comunicar a otros clientes de algún cambio hecho.

El cliente es un usuario del servidor, debe poder conectarse a él y realizar operaciones, además de esperar respuestas del servidor y otras notificaciones de cambios en el chat.

Las operaciones que el usuario debe poder hacer en el chat son:

- **Identificarse:** ingresa un nombre de usuario válido
- **Introducción:** Descripción del proyecto, objetivo y la importancia de conocer esta información.
- **Asignarse un estado:** Escoger entre: ACTIVE, BUSY, AWAY.
- **Ver usuarios:** Puede pedir la lista de usuarios conectados en el servidor.
- **Enviar un texto privado:** Tener mensajes directos con otro usuario.
- **Enviar un texto publico:** Mandar mensajes a todos los usuarios.
- **Crear una sala:** Crear un espacio donde solo los usuarios invitados puedan unirse.
- **Ver usuarios en una sala:** Pedir la lista de usuarios en una sala.
- **Enviar mensaje a una sala:** Mandar mensajes a todos los usuarios en una sala.
- **Enviar mensaje a una sala:** Mandar mensajes a todos los usuarios en una sala.
- **Salir de una sala:** Salir de una sala a la que pertenezca.
- **Desconectarse:** Salir del servidor y de todas las salas a la que pertenezca.

Para que multiples clientes, incluso de diferentes computadoras se comuniquen con el servidor y viceversa se debe usar el protocolo establecido, entonces, usando JSON (JavaScript Object Notation, o notación de objetos en JavaScript), el servidor debe ser capaz de recibir los mensajes del cliente y actuar conforme a ellos y el cliente debe poder enviar los JSON al servidor, conforme a lo que el usuario pida (el usuario no debe escribir los JSON).

Analisis y diseño

Como se explicó en la introducción, el chat se divide en dos partes, **Cliente** y **Servidor**, que deben poder funcionar con cualquier otro cliente y cualquier otro servidor, por lo que creé dos programas distintos, llamados, evidentemente, cliente y servidor:

Cliente

Analisis

El cliente decidí hacerlo en “c#”, porque pensé inicialmente en hacer una interfaz gráfica (que no hice), pero me quedé con él porque es un lenguaje orientado a objetos, lo que me permitió hacer mis distintas clases para formar al cliente, e incluso poder separar la vista del modelo. El cliente es el que representa al usuario y debe tener las operaciones descritas antes, entonces lo que hace el cliente, es básicamente:

- **Conectar al cliente al servidor**

Debe poder recibir y usar los datos del puerto y la dirección que el usuario especifique para conectarse al servidor.

- **Mandar mensajes al servidor**

Recibir las cadenas que el usuario escriba (el usuario no debe escribir el JSON), y “traducirlas” formando el JSON que se enviará al servidor.

- **Recibir mensajes del servidor**

Recibir los JSON que mande el servidor y comunicarlos al cliente de manera comprensible.

- **Comunicar los mensajes al usuario**

Recibir los JSON que mande el servidor y comunicarlos al cliente de manera comprensible.

Diseño

Para el cliente hice 6 distintas clases, cada una con una función diferente:

Main

Método	Descripción	Firma
Main	Este único método solo se encarga de, primero leer la dirección i y el número de puerto y de iniciar la aplicación pasando éstos parámetros, también de inicializar y ejecutar la Vista.	static void Main(string[] args)

Table 1: Métodos de la clase Main

En el main solamente se lee el valor del puerto y se inicializan la Aplicación y la vista.

Aplicación

Todos los métodos de esta clase, excepto Desconectar(), utilizan el modificador de Task “async” y “await” que permiten hacer operaciones asíncronas, la clase se encarga de juntar el lector y generador de mensajes.

Mensaje

Esta clase no tiene metodos, solo sirve para guardar los posibles valores de los JSON.

Método	Descripción	Firma
Conectar	Este método se encarga de, como su nombre lo dice, conectar al cliente al servidor, usando la ip y el puerto pasados como parámetros en el servidor y usa como auxiliar el método RecibirMensajes() para recibir los mensajes que mande el servidor	public async Task Enviar(Mensaje mensaje)
Enviar	Este método se encarga de convertir un mensaje a json, usa el método convierteajson() de la clase Parser.	private async Task RecibirMensajes()
Desconectar	Este método desconecta al cliente	public void Desconectar()

Table 2: Métodos de la clase Aplicacion

Parser

Método	Descripción	Firma
convierteajson	Este método se encarga de recibir un mensaje, de los definidos en la clase Mensaje y regresarlo como una string en la forma de JSON, usando el metodo SerializeObject()	public static string convierteajson(Mensaje m)
convierteamensaje	Este método se encarga de recibir una string s, en forma de JSON y convertirla en un tipo de Mensaje(igual de la clase Mensaje), usando el método DeserializeObject	public static string convierteamensaje(string s)

Table 3: Métodos de la clase Parser

La clase se encarga de “traducir” tanto un Mensaje a una string con forma de JSON y viceversa. Usa la biblioteca “Newtonsoft.Json”, inicialmente iba a utilizar la biblioteca “System.Text.Json,” pero el compilador que utiliza ninja no lo reconocía por lo que tuve que cambiar a este.

Escribe

Método	Descripción	Firma
Traduce	crea los mensajes a partir del json recibido	public static Mensaje Traduce(string s
CrearIdentify	crea el mensaje para la operacion IDENTIFY	private static Mensaje CrearIdentify(string argumentos)
CrearStatus	crea el mensaje para la operacion STATUS	private static Mensaje CrearStatus(string argumentos)
CrearText	crea el mensaje para la operacion TEXT	private static Mensaje CrearText(string argumentos)
CrearPublicText	crea el mensaje para la operacion PUBLICTEXT	private static Mensaje CrearPublicText(string argumentos)
CrearInvita	crea el mensaje para la operacion INVITE	private static Mensaje CrearInvite(string argumentos)
CrearRoomText	crea el mensaje para la operacion ROOMTEXT	private static Mensaje CrearRoomText(string argumentos)
msjAyuda	crea el mensaje de ayuda que explica como usar el programa	private static Mensaje msjAyuda()

Table 4: Métodos de la clase Parser

La clase se encarga de construir los mensajes que se envían al servidor.

Vista

Esta clase es lo que se le muestra al cliente.

Método	Descripción	Firma
Iniciar	Inicia el cliente	public async Task Iniciar()
MostrarMensaje	Este método imprime el mensaje según el caso	private void MostrarMensaje(Mensaje mensaje)
MostrarError	Este método imprime un error si ocurre	private void MostrarError(string error)

Table 5: Métodos de la clase Vista

Servidor

El servidor es el que debe recibir a los clientes, procesar sus mensajes y actuar conforme a ellos:

- **Conectarse a la red**

Debe poder recibir y usar los datos del puerto y la dirección que el usuario especifique para conectarse al servidor. Para esto tengo la clase Servidor, con el método sirve(), el puerto se lee en el Main y se pasa como parámetro al hacer el Servidor

- **Recibir clientes**

Siempre recibir nuevos clientes. Esto igual es en el método sirve que usa el hilo principal para siempre recibir los clientes.

- **Realizar las operaciones pedidas**

Dependiendo del mensaje del cliente debe realizar las operaciones. En la clase servidor, el método sirve, usa el método leeCliente que es el que realiza una operación dependiendo el JSON recibido.

- **Mandar los mensajes**

Después de realizar las operaciones debe mandar respuestas al cliente o mensajes a los demás clientes. En cada método construyo los JSON para enviar al cliente dependiendo la operación, tengo enumeraciones

que hice pensando en hacer más fácil obtener las respuestas y mensajes, pero al final solo la usé en un método.

- **Otras clases**

- **Conexión:** es un mapa para guardar los usernames y su numero de socket.
- **Conexión:** es un mapa para guardar los usernames y su numero de socket.
- **Sala:** Tiene una lista de usuarios y un set de invitados, la sala general es a la que entran todos y luego Servidor tiene un mapa de salas que guarda su nombre y su creador.
- **ListaUsuarios:** es un mapa para guardar a los usuarios junto con su estado, muy util para cuando piden la lista de usuarios en una sala o en el chat general.
- **Bitacora:** Imprime los mensajes para el servidor.
- **Mensaje:** enumeraciones para crear o deseriar los JSON, pero me terminó confundiendo más así que solo las usé en un método.

Implementación

Usé el sistema de construccion meson, ya que es el que el profesor sugirió, se me hizo sencillo de entender y en el README se puede ver como ejecutarse. En el servidor, se ejecuta como `./build/servidor` y luego se pasa el número de puerto. En el cliente se ejecuta como `./build/cliente.exe` y se le pasa, primero, en una línea la dirección ip y en otra línea el puerto.

Conclusiones

Fue un proyecto bastante difícil, pero muy interesante.

Resumen

Durante este proyecto se creó un chat en línea en el que múltiples clientes se puedan conectar y mandar mensajes entre ellos, utilizando otras diversas operaciones dentro de el chat, se tiene una sala general para todos los usuarios y que además cada uno pueda hacer sus salas privadas, invitando a otros usuarios, también podrán solicitar la lista de usuarios conectados, tanto para la sala general como para las que son parte, actualizar su estado, y desconectarse del chat. En este reporte vimos primero una introducción con una descripción del proyecto, luego vimos el proceso de analisis y modelado, hasta llegar a la implementación y luego explicamos las conclusiones.

Fuentes

- Llamas, L. (2024, 24 noviembre). Tasks en “c#”: tareas asíncronas con Task con ejemplos. Luis Llamas. <https://www.luisllamas.es/csharp-tasks/>
- BillWagner. (s.f.). Objetos: creación de instancias de tipos - “c#”. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/object-oriented/objects>